
Advanced signal processing

Exam 1

Name	Initials	Student number	Retning
Alexander Nygaard Thomsen	AT	201911329	E

Due date: 4/10/2026

1 Problem 1

Consider a causal system given by the following system function

$$H(z) = 1 + 3.5z^{-1} + 1.5z^{-2}$$

And assume that the causal system is used as a model for a communication channel that will distort signals passing through it.

1.1 1.

Design an equalizer that can mitigate the distortion The system function is being described as causal, meaning that it cannot depend on future inputs, only on previous inputs $x[n], x[n-1], x[n-2], \dots$. In non-minimum-phase systems, the distortion from systems $H(z)$ can be *undone* by cascading an equalizer, or filter, with the system function, such that when multiplied by its inverse, its effects will be undone. $H(z)H_{inv}(z) = 1$. From the given system function, the zeros and poles can be extracted. A stable and causal system, should have poles inside the unit circle, but the zeros does not need to be. To find out what kind of equalizer we should construct, we can firstly analyze our system to find these.

$$H(z) = 1 + 3.5z^{-1} + 1.5z^{-2} \Rightarrow H(z)z^2 = z^2 + 3.5z^1 + 1.5 \Rightarrow H(z) = \frac{z^2 + 3.5z^1 + 1.5}{z^2}$$

rewritten as

$$\frac{(z+3)(z+0.5)}{z^2}$$

yielding zeroes in: $z = -3, z = -0.5$. Here one zero lies within the unit circle, while the other does not. This is not a problem for the original system.

Our equalizer should ideally be an inverse of the original system, such that

$$H_{inv} = \frac{1}{H(z)} = \frac{A(z)}{B(z)}$$

such that

$$H(z) = \frac{B(z)}{A(z)}$$

however because of this, the zeroes from the original becomes the poles, which now resides outside the unit circle making it unstable. Therefore we must decompose this system into a minimum-phase system and an allpass system.

$$H(z) = H_{min}(z)H_{ap}(z)$$

taking the deduced system function from earlier, this can be rewritten into an expression, that can be made into a system function that will be composed of our minimum-phase and all-pass systems

$$H(z) = \frac{(z+3)(z+0.5)}{z^2} \Rightarrow \frac{z+0.5}{z} \frac{z+3}{z} \Rightarrow (1+0.5z^{-1})(1+3z^{-1})$$

In this case, we have a zero $z = -3$ outside the unit circuit, and $z = \frac{1}{a}, |a| < 1 \Rightarrow a = -\frac{1}{3}$ The composition can be written as:

$$H(z) = H_1(z) \underbrace{(1 - az^{-1})}_{\text{minimum-phase factor}} \underbrace{\frac{z^{-1} - a^*}{1 - az^{-1}}}_{\text{All-pass factor}}$$

$\Downarrow$

$$H(z) = 3(1+0.5z^{-1})(1+\frac{1}{3}z^{-1})\frac{z^{-1}+\frac{1}{3}}{1+\frac{1}{3}z^{-1}}$$

Where $H_1(z) = (1 + 0.5z^{-1})$ and the additional factor of 3, added to the front of $H_1(z)$ is the scale factor, which the magnitude of a.

cleaning up the expression yields:

$$H(z) = 3(1 + 0.5z^{-1})(z^{-1} + \frac{1}{3})$$

All-pass filters affect phase, but not magnitude, therefore we can construct our filter as

$$H_{eq}(z) = \frac{1}{H_{min}} = \frac{1}{3(1 + 0.5z^{-1})(1 + \frac{1}{3}z^{-1})}$$

As mentioned earlier, there is a relationship between the input system, the minimum phase and the all pass function. Since we have the inverse of the minimum phase, we can multiply the equalizer with the input system and then get the all pass function.

$$\begin{aligned} H(z)H_{eq}(z) &= 3(1 + 0.5z^{-1})(1 + \frac{1}{3}z^{-1}) \frac{z^{-1} + \frac{1}{3}}{1 + \frac{1}{3}z^{-1}} \cdot \frac{1}{3(1 + 0.5z^{-1})(1 + \frac{1}{3}z^{-1})} \\ &\Downarrow \\ &= \frac{3(1 + 0.5z^{-1})(z^{-1} + \frac{1}{3})}{3(1 + 0.5z^{-1})(1 + \frac{1}{3}z^{-1})} \\ &\Downarrow \\ &= \frac{(z^{-1} + \frac{1}{3})}{(1 + \frac{1}{3}z^{-1})} = H_{ap}(z) \end{aligned}$$

And this expression is equal to our allpass system.

The magnitude- and phase response is shown below

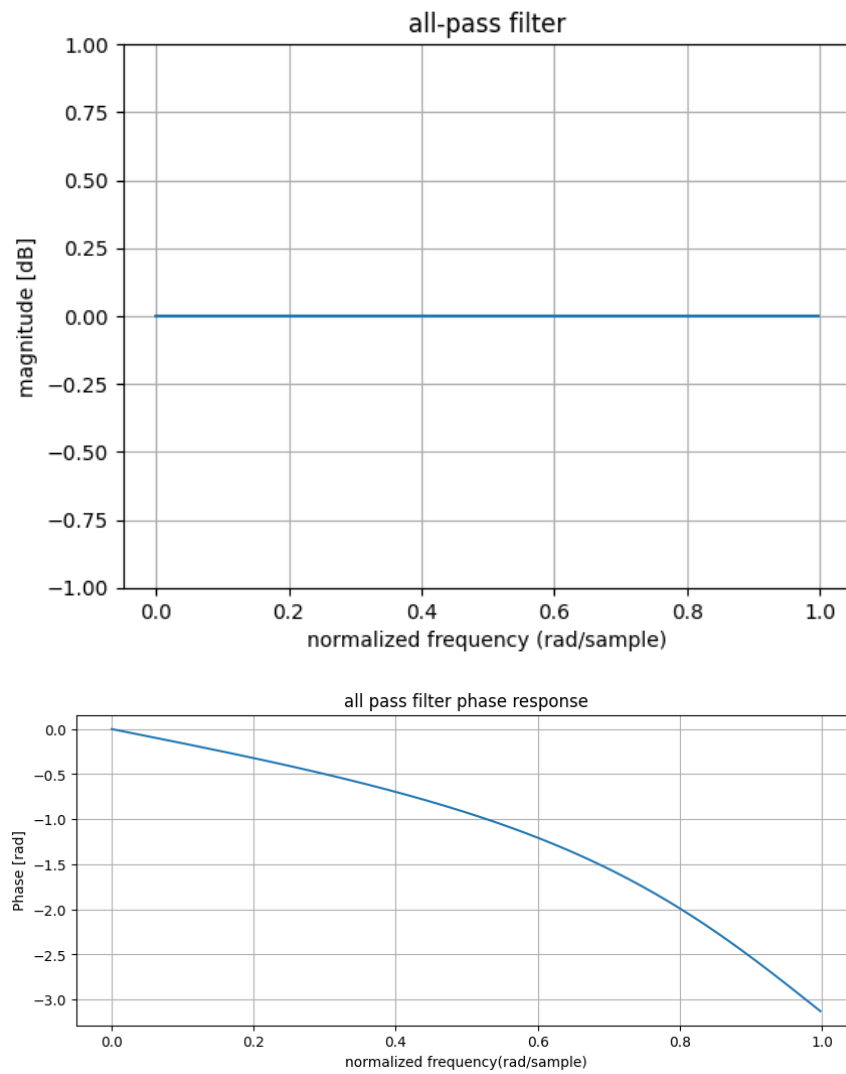


Figure 1: magnitude and phase response of the deduced all-pass filter

1.2 2.

Make a time-domain simulation of the system and the equalizer using a test signal of your choice. Explain the choice of test signal and procedures .. analyze the performance of the equalizer

A good test signal for the system, with the equalizer, would be a squarewave signal as it contains its fundamental frequency, as well as, essentially, infinitely many odd harmonics thereof. We'll select a signal with frequency of $f = 2kHz$ and a sampling rate $f_s = 48kHz$

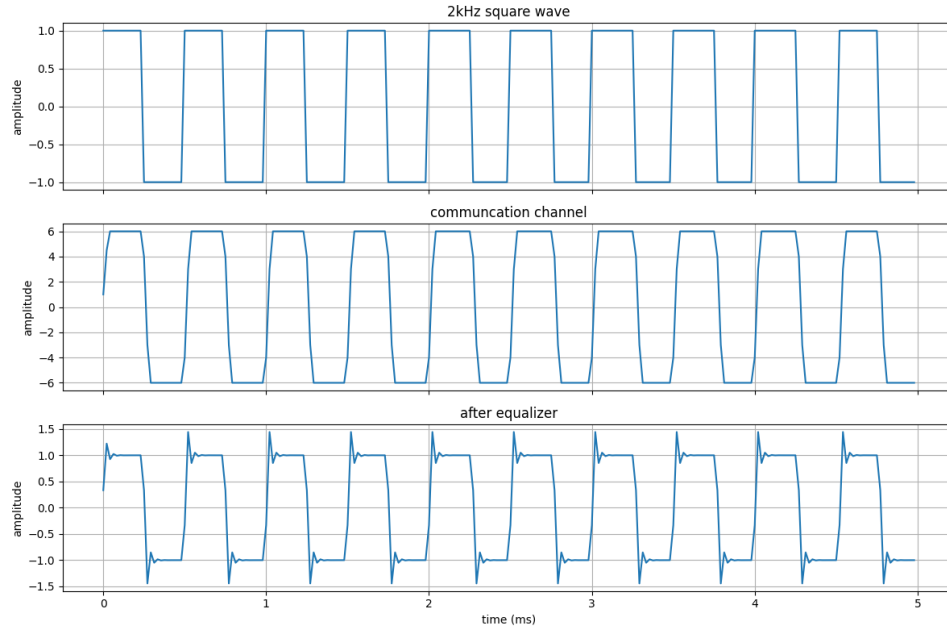


Figure 2: 2kHz testsignal, passed through, first the communication channel and the through the equalizer. Note the slight overshoot of the signal edges from the rise and fall times.

1.3 3.

How are the results affected by the additive noise?

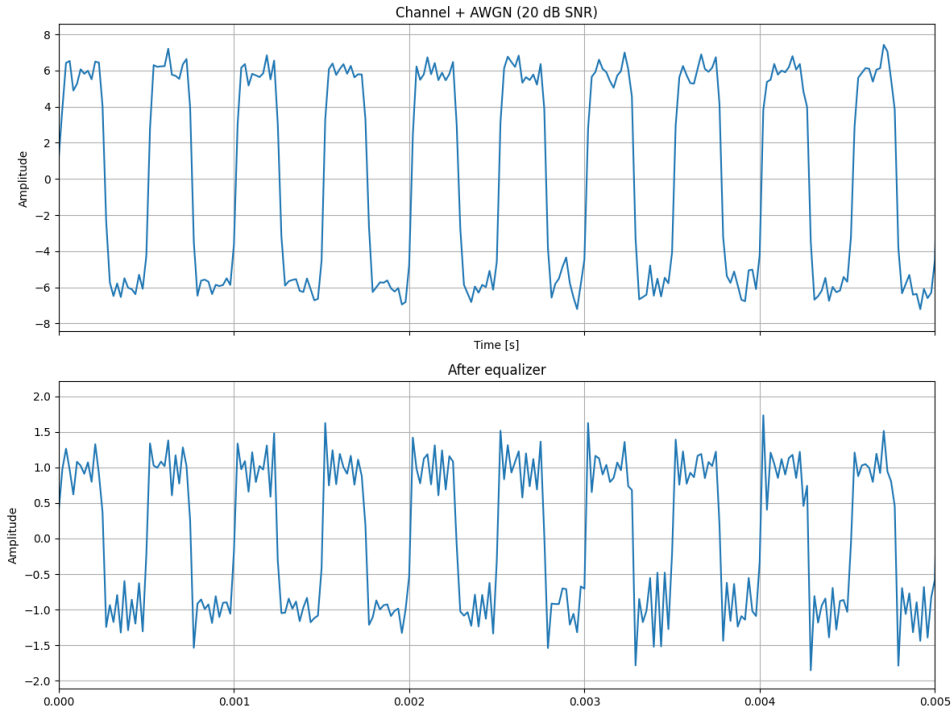


Figure 3: white gaussian noise added after the communication channel, in this case the noise is 20dB SNR. Before the eq, the signal waveform can easily be made out, however after the eq, some sharpness has been retained in the rise and falling edges, however distortion is still present.

2 Problem 2

Consider the random process $x[n]$ created as $x[n] = 4w[n] - 2w[n - 3]$ Where $w[n]$ is unit variance white Gaussian noise.

2.1 1.

Compute an analytic expression for the autocorrelation $r_{xx}[l]$

The definition of the autocorrelation, how a signal correlates to itself, takes the form of

$$r_{xx}[l] = \mathbf{E}\{x[n]x[n-l]\}$$

Inserting the given expression of $x[n]$ into the autocorrelation definition yields a rather length expression:

$$r_{xx}[l] = \mathbf{E}\{(4w[n] - 2w[n-3]) \cdot (4w[n-l] - 2w[n-l-3])\}$$

expanding

$$r_{xx}[l] = \mathbf{E}\{16w[n]w[n-l] - 8w[n]w[n-l-3] - 8w[n-3]w[n-l] + 4w[n-3]w[n-l-3]\}$$

Since $w[n]$ is described as being unit variance white gaussian noise, we also make the assumption that the noise is zero-mean. Unit variance meaning that the variance $\sigma^2 = 1$. Because of this, we can write the definition of the autocorrelation of white noise

$$\mathbf{E}\{x[n]x[n-l]\} = \sigma^2\delta[l]$$

we can then apply this rule to every expectation and move the coefficients to each term, and making makes the difference between each sample index equal to zero. essentially we want the argument of the delta function to become zero and it follows the rule:

$$\delta[l] = \begin{cases} \sigma^2, & \text{if } l = 0 \\ 0, & \text{if } l \neq 0 \end{cases}$$

first term:

$$16\mathbf{E}\{w[n]w[n-l]\} = \delta[l] \Rightarrow 16\delta[l]$$

second term:

$$\begin{aligned} n = n - l - 3 &\Rightarrow l = -3 \Rightarrow \delta[l+3] \\ -8\mathbf{E}\{w[n]w[n-l-3]\} &= \delta[l] \Rightarrow -8\delta[l+3] \end{aligned}$$

third term:

$$\begin{aligned} n - 3 = n - l &\Rightarrow l = 3 \\ -8\mathbf{E}\{w[n-3]w[n-l]\} &= \delta[l] \Rightarrow -8\delta[l-3] \end{aligned}$$

fourth term:

$$\begin{aligned} n - 3 = n - l - 3 &\Rightarrow l = 0 \\ 4\mathbf{E}\{w[n-3]w[n-l-3]\} &= 4\delta[l] \end{aligned}$$

Combining the terms:

$$r_{xx}[l] = 20\delta[l] - 8\delta[l+3] - 8\delta[l-3]$$

This leads to the autocorrelation:

$$r_{xx}[l] = \begin{cases} 20, & l = 0 \\ -8, & l = \pm 3 \\ 0, & \text{otherwise} \end{cases}$$

2.2 2.

Plot the power spectral density of the random process. What can be learned from the plot?

The *PSD* of a stationary random signal, is the fourier transform of the autocorrelation, which shows the distribution of power as a function of frequency. It can be calculated using:

$$S_{xx}(e^{j\omega}) = \sum_{l=-\infty}^{\infty} r_{xx}[l]e^{-jl\omega}$$

If this is applied to r_{xx} from before, we can see that we only have conditions for l in 3 cases, all other cases the summation term becomes zero. Thus it can be expressed as:

$$\begin{aligned} S_{xx}(e^{j\omega}) &= r_{xx}[-3]e^{-j(-3)\omega} + r_{xx}[-0]e^{-j0\omega} + r_{xx}[3]e^{-j3\omega} \\ &\Downarrow \\ &= 20e^{j0\omega} - 8e^{-j(-3)\omega} - 8e^{-j3\omega} \\ &\Downarrow \\ &= 20 - 8e^{j3\omega} - 8e^{-j3\omega} \end{aligned}$$

this can be rewritten using euler identities relating euler complex functions to sinusoids.

$$\begin{aligned} &= 20 - 8(e^{j3\omega} + e^{-j3\omega}) \\ &\Downarrow \\ &= 20 - 8(\cos(3\omega) + j\sin(3\omega) + \cos(3\omega) - j\sin(3\omega)) \\ &\Downarrow \\ &= 20 - 8(2\cos(3\omega)) \\ &\Downarrow \\ &= 20 - 16\cos(3\omega) \end{aligned}$$

which can be plotted:

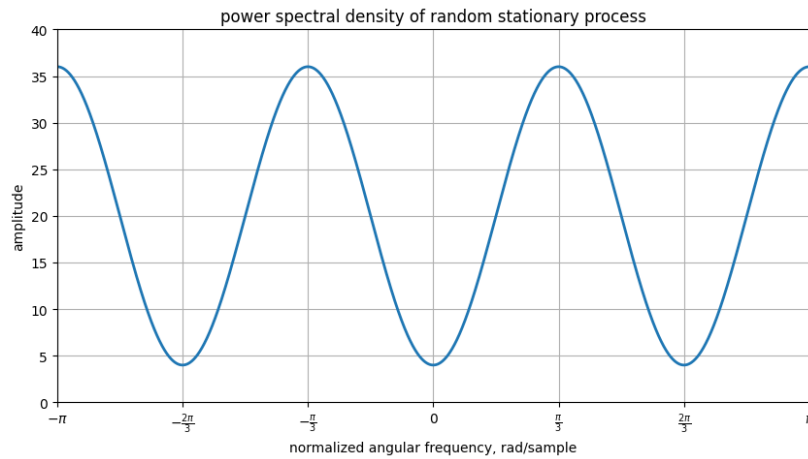


Figure 4: power spectral density depicts a sinusoid, which means it is periodic

2.3 3.

Assume $x[n]$ is filtered through a FIR filter with $H(z) = 5 + z^{-1}$

Compute the analytical expressions for $r_{yx}[l]$ and $r_{yy}[l]$

Firstly, were given the filter in the z-domain, where the filter essentially tells us "take the current samples and multiply by 5 and add it to the previous input sample" which can be inverse z-transformed to yield:

$$h[l] = 5\delta[l] + \delta[l-1]$$

It is then possible to use the formula described in (13.100), and because we found r_{xx} earlier

$$r_{yx}[l] = h[l] * r_{xx}[l]$$

Using the impulse-convolution rule and substituting $h[l]$ and r_{xx} into the convolution using $\delta[l-a] * \delta[l-b] = \delta[l-(a+b)]$, yielding six impulse terms.

$$r_{yx}[l] = (5\delta[l] + \delta[l-1]) * (20\delta[l] - 8\delta[l+3] - 8\delta[l-3])$$

$$\Downarrow$$

$$5\delta[l] * 20\delta[l] = 100\delta[l]$$

$$5\delta[l] * (-8\delta[l-3]) = -40\delta[l-3]$$

$$5\delta[l] * (-8\delta[l+3]) = -40\delta[l+3]$$

$$\delta[l-1] * 20\delta[l] = 20\delta[l-1]$$

$$\delta[l-1] * (-8\delta[l-3]) = -8\delta[l-4]$$

$$\delta[l-1] * (-8\delta[l+3]) = \delta[l-1] * (-8\delta[l-(1+(-3))]) = -8\delta[l+2]$$

summing all terms:

$$r_{yx}[l] = 100\delta[l] + 20\delta[l-1] - 40\delta[l-3] - 40\delta[l+3] - 8\delta[l-4] - 8\delta[l+2]$$

To find r_{yy} , equation 13.111 from the book can be used

$$R_{yy} = H(Z)H\left(\frac{1}{Z}\right)R_{xx}(Z)$$

This requires us to use $R_{xx}(Z)$, meaning a z-transform of $r_{xx}(l)$ is required. The functions then becomes:

$$H(Z) = 5 + Z^{-1}$$

$$H\left(\frac{1}{Z}\right) = \frac{1}{5 + Z^{-1}} \Rightarrow 5 + \frac{1}{Z} = 5 + Z$$

For Z-transformation:

$$Z(\delta(l-k)) = z^{-k}$$

$$r_{xx}[l] = 20\delta[l] - 8\delta[l+3] - 8\delta[l-3]$$

$$\Downarrow Z - trans$$

$$R_{xx}[Z] = 20 - 8Z^3 - 8Z^{-3}$$

Combining all the expressions:

$$R_{yy} = (5 + Z^{-1})(5 + Z)(20 - 8Z^3 - 8Z^{-3})$$

Solved using CAS tool

$$\Downarrow$$

$$R_{yy}[Z] = (-40z^4) - 208z^3 - 40z^2 + 100z + 520 + 100z^{-1} - 40z^{-2} - 208z^{-3} - 40z^{-4}$$

Inverse Z-transform to get $r_{yy}[l]$

$$r_{yy}[l] = (-40\delta(l+4)) - 208\delta(l+3) - 40\delta(l+2) + 100\delta(l) + 520 + 100\delta(l-1) - 40\delta(l-2) - 208\delta(l-3) - 40\delta(l-4)$$

3 problem 3

Let a signal and interpolation kernel be given by

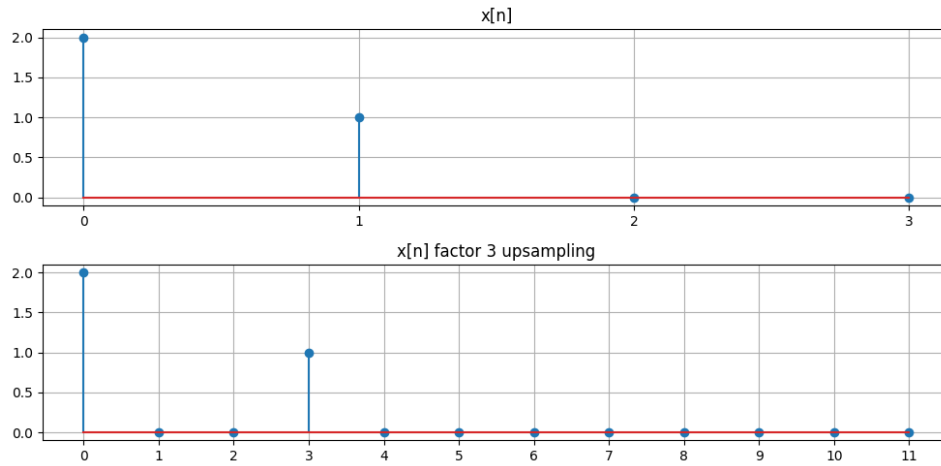
$$x[n] = \begin{cases} 2, & n = 0 \\ 1, & n = 1 \\ 0, & \text{elsewhere} \end{cases}$$

$$g[n] = \begin{cases} -1, & n = -1 \\ 3, & n = 0 \\ -1, & n = 1 \\ 0, & \text{elsewhere} \end{cases}$$

3.1 1.

Upsample $x[n]$ by a factor 3 and sketch the upsampled signal. Here we upsample the signal by inserting two zeroes between every original sample. There the first sample is 2 and the second is 1, we then insert two zeroes in between.

$$x[n] = [2, 1, 0, 0] \Rightarrow [2, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0]$$



3.2 2.

Compute the interpolated values $x_1[1]$ and $x_1[2]$ using first linear interpolation and second the proposed $g[n]$ kernel. To implement the linear interpolation scheme, we use the formula for the general interpolation:

$$x_I[n] = \sum_{m=-\infty}^{\infty} x[m]g_r[n - mI]$$

In our case we only have $x[0] = 2$ and $x[1] = 1$, so only $m = 0$ and 1 contributes. And uses the kernel

$$g_{lin}[n] = \begin{cases} 1 - \frac{|n|}{l}, & -l < n < l \\ 0, & \text{otherwise} \end{cases}$$

Substituting this into the interpolation formula:

$$x_I[n] = 2g_r[n] + 1 \cdot g_r[n - 3]$$

The ideal interpolation is from negative to positive infinity, however this is impractical and impossible. The practical approach is to only do it from all the original samples $x[n]$, left and right of the current sample. Thus when we build our linear interpretation kernel from the cases above, we have to extend the range in both directions. Our kernel values become:

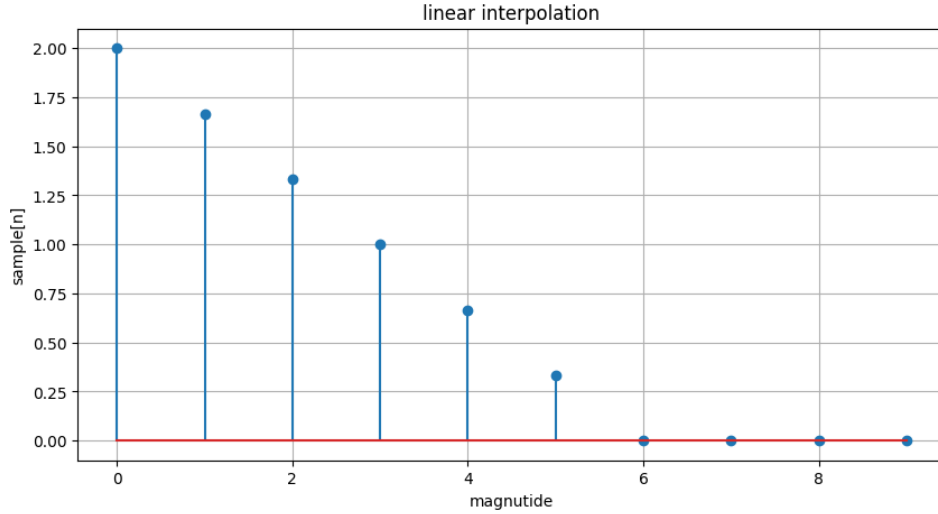
$$g_{lin}[0] = 1, \quad g_{lin}[\pm 1] = 1 - \frac{1}{3} = \frac{2}{3}, \quad g_{lin}[\pm 2] = 1 - \frac{2}{3} = \frac{1}{3}, \quad g_{lin}[\pm 3] = 0$$

leading to:

n	-3	-2	-1	0	1	2	3
$g_{lin}[n]$	0	$\frac{1}{3}$	$\frac{2}{3}$	1	$\frac{2}{3}$	$\frac{1}{3}$	0

Our original sequence was $x[n]$, and when linearly interpolated, becomes:

$$\begin{aligned} x_I[0] &= 2, & x_I[1] &= \frac{5}{3}, \\ x_I[2] &= \frac{4}{3}, & x_I[3] &= 1 \\ x_I[4] &= \frac{2}{3}, & x_I[5] &= \frac{1}{3}, & x_I[6] &= 0 \end{aligned}$$



So $x_I[1]$ and $x_I[2]$ in linear interpolation can be written as

$$x_I[1] = \sum_{m=0}^1 x[m] \cdot g_{lin}[1 - m \cdot 3] = x[0]g_{lin}[1 - (0 \cdot 3)] + x[1]g_{lin}[1 - (1 \cdot 3)] = 2\left(\frac{2}{3}\right) + 1\left(\frac{1}{3}\right) = \frac{5}{3}$$

$$x_I[2] = \sum_{m=0}^1 x[m] \cdot g_{lin}[2 - m \cdot 3] = x[0]g_{lin}[2 - (0 \cdot 3)] + x[1]g_{lin}[2 - (1 \cdot 3)] = 2\left(\frac{1}{3}\right) + 1\left(\frac{2}{3}\right) = \frac{4}{3}$$

As for the proposed interpolation kernel $g[n] \Rightarrow g_p[n]$, renamed for convenience. Starting from the same interpolation formula:

$$x_p[n] = 2g_p[n] + g_p[n - 3]$$

$$x_p[-1] = 2g_p[-1] + g_p[(-1) - 3] = 2(-1) + 0 = -2, \quad x_p[0] = 2g_p[0] + g_p[0 - 3] = 2(3) + 0 = 6,$$

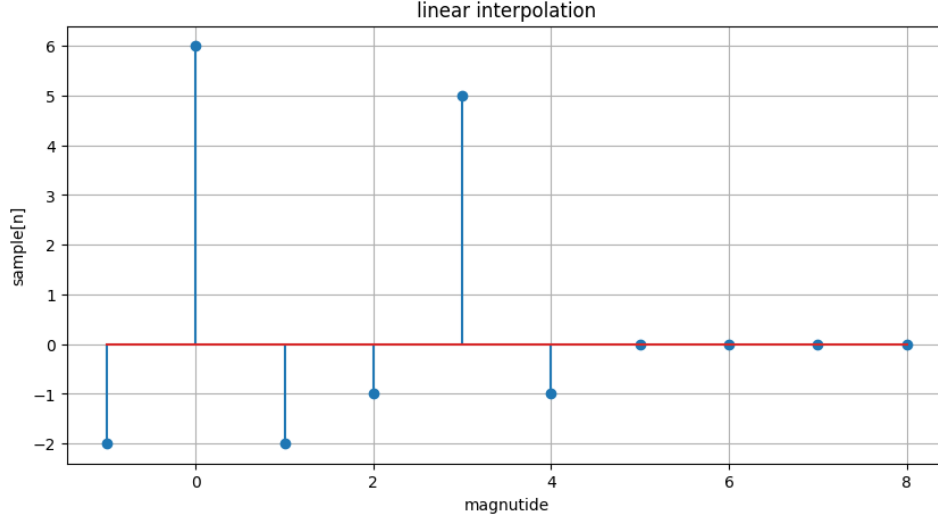
$$x_p[1] = 2g_p[1] + g_p[1 - 3] = 2(-1) + 0 = -2, \quad x_p[2] = 2g_p[2] + g_p[2 - 3] = 2(0) + (-1) = -1,$$

$$x_p[3] = 2g_p[3] + g_p[3 - 3] = 2(0) + 3 = 5, \quad x_p[4] = 2g_p[4] + g_p[4 - 3] = 2(0) + (-1) = -1$$

$$x_p[5] = 2g_p[5] + g_p[5 - 3] = 0 + 0 = 0,$$

leading to:

$$x_I[-1] = -2, \quad x_I[0] = 6, \quad x_I[1] = -2, \quad x_I[2] = -1, \quad x_I[3] = 5, \quad x_I[4] = -1, \quad x_I[5] = 0,$$



And $x_I[1]$ and $x_I[2]$ with the proposed kernel, written explicitly:

$$x_I[1] = \sum_{m=0}^1 x[m]g_p[1 - 3m] = x[0]g_p[1 - 3(0)] + x[1]g_p[1 - 3(1)] = 2(-1) + 1(0) = -2$$

$$x_I[2] = \sum_{m=0}^1 x[m]g_p[2 - 3m] = x[0]g_p[2 - 3(0)] + x[1]g_p[2 - 3(1)] = x[0]g_p[2] + x[1]g_p[-1] = 2(0) + 1(-1) = -1$$

3.3 3.

The two interpolations, behave quite differently. The linear interpolation, obviously looks just like a linear function, whereas the proposed almost looks like a *sinc* function. The proposed one, does not preserve any of the original values, whereas the linear does. As in, it gains them quite drastically and also attenuates quite hard. So it could be said that the proposed does not perform a proper interpolation as it does not preserve the original samples in magnitude, only in signal shape.

4 problem 4

This problem will be conducted using python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import decimate, resample_poly
% load sample in and make a sample axis
x = np.loadtxt("SampleRateData.dat")
n = np.arange(len(x))
%downsample by 2, using decimate, since its decimation apply a fir filter ,
M = 2
y = decimate(x, M, ftype='fir ', zero_phase = True)
ny = np.arange(len(y))
%upsample by 2, using resample_poly, with an upfactor of 2 and a downfactor of 1
L = 2
z_up = np.zeros(len(y) * L)
z = resample_poly(y,up=L, down=1)
nz = np.arange(len(z))

%for ease of readability, plotting functionality has been omitted.
```

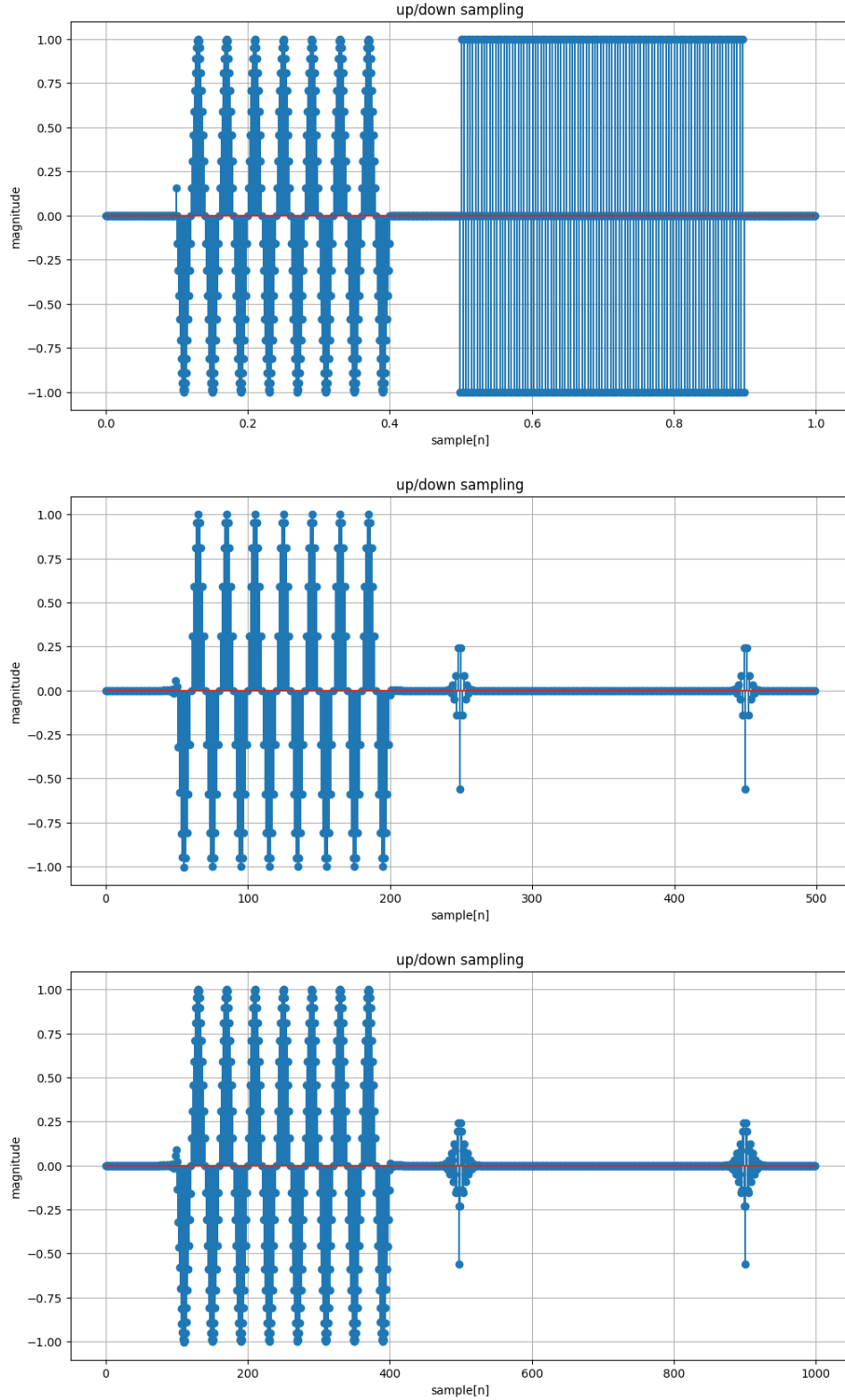


Figure 5: top picture is the samples loaded into python, with no processing. The middle is after decimation, note that there are small changes the signal, that is because the decimation by a factor of two, brought with it a low pass filter, as the values suddenly change, the filter has to react, however it cannot do so instantly. The last graph is an interpolation by 2. It reconstructs, by estimation, some of the data lost to the decimation, however this is approximated, as it can never be recovered exactly.

5 problem 5

Consider a two-channel filter bank where it is intended to operate it as a quadrature mirror filter with $H(z) = 1 + \frac{1}{2}Z^{-1}$

5.1 1.

Are $H_0(z)$ and $H_1(z)$ power complementary?

For two filters to be power complementary, they must satisfy the condition that

$$|H_0(e^{j\omega})|^2 + |H_1(e^{j\omega})|^2 = 1$$

, however this also applies to filters that takes the form of $H(z)$ here the condition is:

$$H(z)H(z^{-1}) + H(-z)H(-z)^{-1} = 1$$

In our case $H_0(z) = H(z) = 1 + \frac{1}{2}Z^{-1}$ as for $H(z^{-1})$ this we can replace Z with $Z^{-1} \Rightarrow H(Z^{-1}) = 1 + \frac{1}{2}(Z^{-1})^{-1} = 1 + \frac{1}{2}Z$
 $H(-z)$ and $H(-z^{-1})$ is constructed as:

$$H(-z) = 1 + \frac{1}{2}(-Z)^{-1} \Rightarrow 1 - \frac{1}{2}Z^{-1}$$

$$H(-z)^{-1} = 1 + \frac{1}{2}(-Z^{-1})^{-1} \Rightarrow 1 - \frac{1}{2}Z$$

Put all the Substitutions together yields:

$$\begin{aligned} H(z)H(z^{-1}) + H(-z)H(-z)^{-1} &= (1 + \frac{1}{2}Z^{-1})(1 + \frac{1}{2}Z) + (1 - \frac{1}{2}Z^{-1})(1 - \frac{1}{2}Z) \\ &\Downarrow \\ &= (1 + \frac{1}{2}Z + \frac{1}{2}Z^{-1} + \frac{1}{4}) + (1 - \frac{1}{2}Z - \frac{1}{2}Z^{-1} + \frac{1}{4}) \\ &\quad \frac{5}{4} + \frac{5}{4} = \frac{10}{4} = \frac{5}{2} \end{aligned}$$

In other words $\frac{5}{2} \neq 1$ therefore $H_0(z)$ and $H_1(z)$ are not power complementary

5.2 2.

Will the choice of $H(z)$ lead to an aliasing-free filter bank?

The purpose of the quadrature mirror filter is to provide cancellation of output aliasing , but the reconstruction property is also approximate. The system function of a alias-free QMF is given by.

$$T(z) = H^2(z) - H^2(-z)$$

from ealier we know those two functions:

$$\begin{aligned} &(1 + \frac{1}{2}Z^{-1})^2 - (1 - \frac{1}{2}Z^{-1})^2 \\ &\Downarrow \\ &(1 + \frac{1}{2}Z^{-1})(1 + \frac{1}{2}Z^{-1}) - (1 - \frac{1}{2}Z^{-1})(1 - \frac{1}{2}Z^{-1}) \\ &\Downarrow \\ &(1 + Z^{-1} + \frac{1}{4}z^{-2}) - (1 - Z^{-1} + \frac{1}{4}z^{-2}) \\ &\Downarrow \\ &1 + Z^{-1} + \frac{1}{4}z^{-2} - 1 + Z^{-1} - \frac{1}{4}z^{-2} \\ &\Downarrow \\ &T(z) = 2Z^{-1} \end{aligned}$$

This expression has a gain of 2 and a delay of 1 sample. This means that the choice of $H(z)$ will lead to an alias-free filter bank.

5.3 3.

Discuss the choice of $H(z)$ for a quadrature mirror filter bank. The idea behind the QMF was to implement it to implement it using the filter banks, designed specifically to combat aliasing errors. If $H(z)$ is limited to have a linear-phase then it would have a symmetric impulse response.

6 problem 6

6.1 1.

Using python, the signal is read in, thereafter mean and power is calculated

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import correlate, correlation_lags

x = np.loadtxt("dataset.dat")
N = len(x)

print("mean:", np.mean(x))
print("power:", np.mean(x**2))

x0 = x - np.mean(x) #remove the mean from the signal
#correlate(x,y), we correlate the first signal with itself
rxx = correlate(x0,x0,mode="full") / N #correlation gives a sum of products, dividing turn
#same as correlate(x,y), correlate the two function inputs.
lags = correlation_lags(N,N, mode="full")

#page 187, "meaningfull measure of similarity for small lags -> 20%"
max_lag = 400
#keep only samples that are within the bounds of a given lag
mask = (lags >= -max_lag) & (lags <= max_lag)

#white noise signal
P = np.mean(x0**2) #compute mean power from the datasett
w = np.random.normal(
    loc=0,          #noise has a mean of 0
    scale=np.sqrt(P), #std is square root of power
    size=N          #amount of samples
)
```

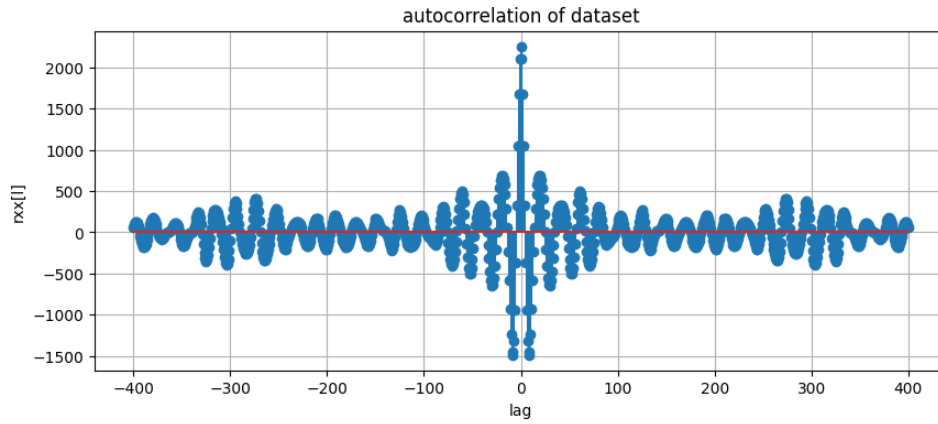


Figure 6: Above is the plot of the autocorrelated data from the dataset. A sharp center can be seen, meaning that there is a point where there is a strong correlation between itself, however there is also clear sidelobes, that could indicate some period elements in the dataset, meaning a slightly stronger correlation elsewhere aswell.

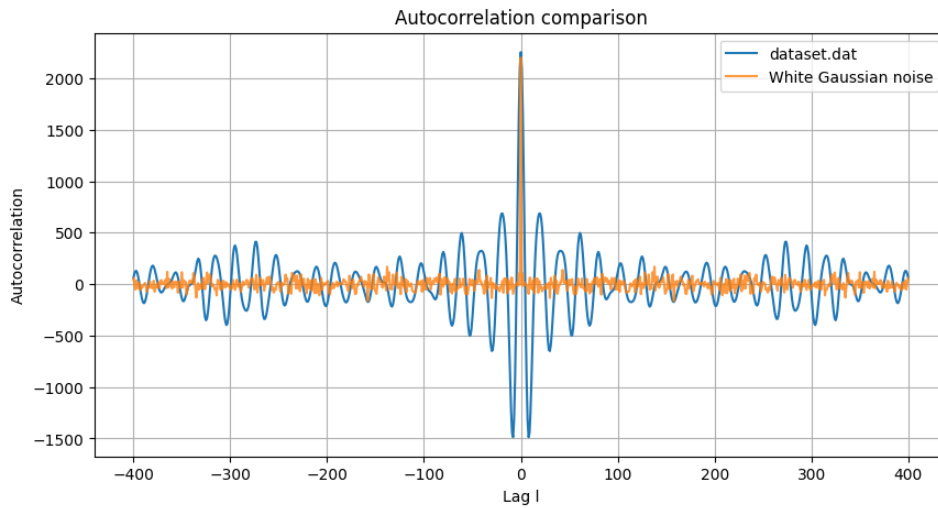


Figure 7: When comparing the autocorrelation of white noise to the dataset signal, it is clear that there is a singular place where the signal is a perfect match, so the autocorrelation of a white noise gaussian signal has a clear maximum at lag 0.